

Project Zenith — Complete Deployment Guide

This guide takes you from **zero** to a **live, public URL** for Project Zenith, even if you have **never deployed a full-stack application before**. Follow it top to bottom. Every click, every field, and every value you need to type is spelled out.

What you are deploying

Project Zenith is a **monorepo** — one GitHub repository containing two separate applications:

Part	Tech	Lives in folder	Deploys to	Result
Backend (API gateway)	Node.js · Express · TypeScript	<code>backend/</code>	Render	<code>https://your-backend.onrender.com</code>
Frontend (website)	Next.js · React · TypeScript	<code>frontend/</code>	Vercel	<code>https://your-app.vercel.app</code>
Code host	Git	whole repo	GitHub	source of truth for both

Order matters. Deploy the **backend first** (Render), copy its URL, then deploy the **frontend** (Vercel) and point it at the backend. Part 5 explains the reconnection step.

The big picture

You → Vercel (frontend website) —HTTPS→ Render (backend API) → NASA
/ Open-Meteo /
https://your-app.vercel.app https://your-backend
Celestrak / Gemini / ...
onrender.com

Both auto-deploy whenever you push to GitHub.

Table of Contents

- [Part 1 — Pre-Deployment Checklist](#)
- [Part 2 — Backend Deployment \(Render\)](#)
- [Part 3 — Verify the Backend](#)
- [Part 4 — Frontend Deployment \(Vercel\)](#)
- [Part 5 — Connect Frontend to Backend](#)
- [Part 6 — Post-Deployment Testing](#)
- [Part 7 — Common Deployment Errors](#)
- [Part 8 — Production Checklist](#)
- [Appendix — Accounts & API Keys You May Need](#)

Part 1 — Pre-Deployment Checklist

Do **all** of these on your own computer **before** touching Render or Vercel. Tick each box.

1.1 Push the latest code to GitHub

- ☐ You have a [GitHub](#) account.
- ☐ The project is a Git repository (`git status` works inside `d:\Projects\project-zenith`).
- ☐ Commit everything and push:

```
git add .
git commit -m "Prepare for deployment"
git push origin main
```

- ☐ Open your repository on [github.com](#) and confirm you can see the `frontend/` and `backend/` folders.

1.2 Verify the README

- ☐ `README.md` exists at the repo root and renders correctly on GitHub.

1.3 Verify `package.json` scripts

Open each file and confirm the scripts. **These exact scripts are what Render and Vercel will run** — we inspected them; do not change them.

- ☐ `backend/package.json` — note there is **no build script**; the app runs TypeScript directly with `tsx`:

```
"scripts": {
  "dev": "tsx watch src/server.ts",
  "start": "tsx src/server.ts"    // ← Render will run this
}
```

- ☐ `frontend/package.json`:

```
"scripts": {
  "dev": "next dev",
  "build": "next build",    // ← Vercel will run this
  "start": "next start",
  "postinstall": "node scripts/copy-cesium-assets.mjs" // ← runs
  automatically, copies Cesium assets
}
```

1.4 Verify `.env.example` files exist (templates, no secrets)

- ☐ `backend/.env.example` exists.
- ☐ `frontend/.env.local.example` exists.

1.5 Ensure real `.env` files are NOT committed

Your real keys must never go to GitHub. Confirm they are ignored:

```
git check-ignore backend/.env frontend/.env.local
```

- ☐ Both paths print back (meaning they are ignored). If nothing prints, **stop** — your `.gitignore` is not protecting them. (In this repo, `.gitignore` already ignores `.env` and `.env.*` except the `.env.example` templates, so you are covered.)
- ☐ On GitHub, confirm you do **not** see `backend/.env` or `frontend/.env.local` in the file list.

1.6 Verify the build/run succeeds locally

Run these locally so you don't debug on the cloud:

Backend (terminal 1):

```
cd backend
npm install
npm start
```

- ☐ You see `Server running on port 8000`.
- ☐ Visiting `http://localhost:8000/api/health` returns `{"status":"OK", ...}`.

Frontend (terminal 2):

```
cd frontend
npm install          # postinstall copies Cesium assets automatically
npm run build        # this is the exact command Vercel runs – it MUST pass
```

- ☐ `npm run build` finishes with **no errors** (✓ `Compiled successfully`).
- ☐ `npm run dev` then opening `http://localhost:3000` shows the globe.

✓ If all boxes are ticked, you are ready to deploy.

Part 2 — Backend Deployment (Render)

Render will host the Express API. Because the repo is a monorepo, the single most important setting is **Root Directory** = `backend` — without it Render tries to build the whole repo and fails.

Step 1 — Create a Render account

- Go to <https://render.com> and click **Get Started**.
- Choose **Sign up with GitHub** (easiest — it links your code automatically).

 Screenshot placeholder: Render sign-up page.

Step 2 — Connect GitHub

1. Render asks to connect to GitHub. Click **Connect GitHub**.
2. GitHub opens an authorization page. Click **Authorize Render**.

 Screenshot placeholder: GitHub "Authorize Render" page.

Step 3 — Authorize the repository

1. GitHub asks which repositories Render may access.
2. Choose **Only select repositories** → pick **project-zenith** → **Install / Save**. (Or "All repositories" if you prefer.)

 Screenshot placeholder: GitHub repository access selection.

Step 4 — Create a Web Service

1. In the Render dashboard click **New +** (top right) → **Web Service**.

 Screenshot placeholder: Render "New +" menu.

Step 5 — Select the repository

1. Find **project-zenith** in the list → click **Connect**.

Step 6 — Branch selection

1. **Branch:** select **main** (the branch you pushed in Part 1).

Step 7 — Root Directory & Environment (Runtime)

Render now shows a configuration form. Fill it exactly:

Field	Value to enter	Why
Name	zenith-backend (any name)	Becomes part of your URL: zenith-backend.onrender.com
Language / Runtime	Node	It's a Node.js app
Root Directory	backend	⚠ Critical — tells Render the app lives in the backend/ folder, not the repo root
Branch	main	From Step 6

 Screenshot placeholder: Render service configuration form.

Step 8 — Build Command

Enter **exactly**:

```
npm install --include=dev
```

Why this exact command (do not just use `npm install`): the backend has **no compile/build step** — it runs TypeScript directly at runtime using `tsx`. But `tsx` and `typescript` live in `devDependencies`. Some hosts set `NODE_ENV=production`, which makes a plain `npm install` **skip devDependencies**, and the app would then crash with `tsx: not found`. The `--include=dev` flag forces those tools to install so the Start Command can run. There is **no `npm run build`** for this backend — do not invent one.

Step 9 — Start Command

Enter **exactly**:

```
npm start
```

This runs the real script from `backend/package.json`: `tsx src/server.ts`, which boots the Express server.

Step 10 — Region

1. **Region:** pick the one closest to your users/judges (e.g. **Singapore** for India/Asia, **Oregon/Ohio** for the US, **Frankfurt** for Europe). Any region works; closer = lower latency.

Step 11 — Free vs Paid (Instance Type)

Plan	Cost	Behavior	Use for
Free	\$0	Sleeps after ~15 min of no traffic; first request after sleeping takes ~30–60 s to wake	Hackathon demo (fine, just warm it up first — see Part 7)
Starter (paid)	~\$7/mo	Always on, no cold starts	If you want zero wake-up delay during judging

For a hackathon, **Free** is acceptable. Just hit the backend URL once a minute before your demo to keep it awake.

Step 12 — Auto Deploy

- Set **Auto-Deploy: Yes** (default). Every `git push` to `main` will automatically redeploy the backend.

Step 13 — Health Check Path

Scroll to **Advanced** → **Health Check Path** and enter:

```
/api/health
```

This repo exposes `GET /api/health`, which returns `{"status":"OK", ...}`. Render pings it to confirm the service is alive and to decide if a deploy succeeded.

Step 14 — Port handling (you do NOT set a port)

Do not hardcode a port. Render injects a `PORT` environment variable at runtime, and this backend already reads it:

```
const PORT = process.env.PORT || 8000; // backend/src/server.ts
server.listen(PORT, ...)
```

So in production the server listens on whatever port Render assigns, and falls back to `8000` locally. **You do not need to add a `PORT` variable yourself** — Render handles it.

Step 15 — Environment Variables

Scroll to the **Environment Variables** section and click **Add Environment Variable** for each row below. We inspected the backend source — these are **the only variables it reads**.

Variable	Purpose	Required?	Example value	Sensitive?
GEMINI_API_KEY	Enables live AI Sky Narration via Google Gemini. Without it, narration still works using a built-in templated fallback from the same real numbers.	Optional (recommended)	AIzaSy...	Yes — secret
N2YO_API_KEY	Enables real ISS visible-pass time windows . Without it, the ISS still shows real position/altitude/velocity; only the "next pass" event is omitted.	Optional	ABCDEF-GHIJKL-MNOPQR-1A2B	Yes — secret
PORT	Port the server listens on. Do NOT set this — Render provides it automatically.	No (auto)	(leave unset)	No

Notes:

- **No Cesium key on the backend.** Cesium runs entirely in the browser (frontend). Do **not** add a Cesium variable here.
- **No CORS / frontend-URL variable needed.** The backend enables open CORS (`app.use(cors())`), so your Vercel site can call it out of the box. (You can tighten this later in code if you want.)
- **No database / Redis variable.** Caching is in-process; nothing to configure.
- Where to get the optional keys: see the [Appendix](#).

 Screenshot placeholder: Render environment-variables section with GEMINI_API_KEY and N2YO_API_KEY.

Step 16 — Deploy

1. Click **Create Web Service** (or **Deploy Web Service**).
2. Render starts building. Wait for the status to turn **Live** (green). First deploy takes a few minutes.
3. Copy your backend URL from the top of the page, e.g. <https://zenith-backend.onrender.com>. You'll need it in Part 4/5.

 Screenshot placeholder: Render service page showing "Live" status and the .onrender.com URL.

Step 17 — How to read logs

- On the service page, open the **Logs** tab.
- A healthy boot shows: **Server running on port 10000** (Render's injected port).
- Logs are also where every `[CACHE HIT]/[CACHE MISS]` and `[TIMING]` line appears at runtime.

Step 18 — How to debug a failed deployment

1. Open the **Logs / Events** tab and read the **last error line** — it almost always names the cause.
2. Match it to [Part 7](#). The most common first-deploy failures:
 - `tsx: not found` → your Build Command isn't `npm install --include=dev` (Step 8).
 - `Cannot find module './app'` or path errors → **Root Directory** isn't `backend` (Step 7).
 - Build "succeeds" but service is "unhealthy" → wrong **Health Check Path** (must be `/api/health`).
3. Fix the setting (**Settings** tab) and click **Manual Deploy** → **Deploy latest commit**.

Part 3 — Verify the Backend

Before deploying the frontend, confirm the API works. Replace `YOUR_BACKEND` with your real Render URL (e.g. zenith-backend.onrender.com).

 On the **Free** plan the first request may take ~30–60 s while the service wakes — that's normal.

3.1 Health check (browser)

Open in your browser:

```
https://YOUR_BACKEND/api/health
```

Expected: `{"status":"OK","uptime":...,"timestamp":"...","service":"Project Zenith Backend"}`

3.2 Health check (curl)

```
curl https://YOUR_BACKEND/api/health
```

3.3 Full Celestial Report (the main endpoint)

Path format is `/api/report/<lat>/<lng>?t=<timeline>`. Example for Kolkata, "now":

```
curl "https://YOUR_BACKEND/api/report/22.5726/88.3639?t=now"
```

Expected: a large JSON object containing `score`, `objects`, `events`, and `narration`. (Cold runs can take 10–12 s the first time; cached after.)

3.4 Location search

```
curl "https://YOUR_BACKEND/api/location/search?q=Tokyo"
```

Expected: a JSON list of matching places with `lat/lng`.

3.5 Reverse geocode (globe click)

```
curl "https://YOUR_BACKEND/api/location/35.68/139.65"
```

Expected: a JSON location object for those coordinates.

3.6 A few more (optional)

```
curl "https://YOUR_BACKEND/api/narrate?lat=22.57&lng=88.36&t=now&name=Kolkata" #  
AI narration  
curl "https://YOUR_BACKEND/api/satellite/iss"  
# live ISS position
```

✓ If `/api/health` and `/api/report/...` return JSON, your backend is good. Proceed.

Part 4 — Frontend Deployment (Vercel)

Vercel hosts the Next.js website. Like Render, the key monorepo setting is **Root Directory** = `frontend`.

Step 1 — Create a Vercel account

1. Go to <https://vercel.com> → **Sign Up**.
2. Choose **Continue with GitHub** → **Authorize Vercel**.

 Screenshot placeholder: Vercel sign-up with GitHub.

Step 2 — Import the repository

1. On the Vercel dashboard click **Add New...** → **Project**.

- Find **project-zenith** → click **Import**. (If it's not listed, click **Adjust GitHub App Permissions** and grant access to the repo.)

 Screenshot placeholder: Vercel "Import Git Repository" list.

Step 3 — Framework detection

- Vercel should auto-detect **Next.js** and show its logo. If it does, leave **Framework Preset = Next.js**.

Step 4 — Root Directory

- Find **Root Directory** → click **Edit** → select **frontend**. This is essential — the Next.js app is in the **frontend/** folder, not the repo root.

 Screenshot placeholder: Vercel Root Directory selector set to "frontend".

Step 5 — Build & Output settings (leave as defaults)

With Framework = Next.js and Root Directory = **frontend**, Vercel auto-fills the correct values. **Leave the "Override" toggles OFF** so these defaults apply:

Setting	Value (auto)	Notes
Install Command	<code>npm install</code>	Triggers <code>postinstall</code> , which copies Cesium assets into <code>public/cesium</code>
Build Command	<code>next build</code> (<code>npm run build</code>)	The exact script from <code>frontend/package.json</code>
Output Directory	<code>.next</code>	Next.js default — do not change

Do not set a custom Output Directory. Next.js on Vercel manages this automatically.

Step 6 — Environment Variables

Click **Environment Variables** and add each row below. We inspected the frontend source — these are the variables it reads. All public ones must start with `NEXT_PUBLIC_` (that prefix is what makes Next.js expose them to the browser).

Variable	Purpose	Required?	Value to enter	Sensitive?
<code>NEXT_PUBLIC_API_BASE_URL</code>	The address of your Render backend . The website calls this for all data.	Yes	Your Render URL, e.g. <code>https://zenith-backend.onrender.com</code> (no trailing slash)	No
<code>NEXT_PUBLIC_DATA_SOURCE</code>	<code>live</code> routes through the backend (with automatic	Yes	<code>live</code>	No

Variable	Purpose	Required?	Value to enter	Sensitive?
	mock fallback). <code>mock</code> forces a fully-offline demo (no backend needed).			
<code>NEXT_PUBLIC_CESIUM_ION_TOKEN</code>	Cesium ion token for high-resolution imagery/terrain. If omitted, the globe falls back to bundled offline imagery (still works).	Optional	<code>eyJhbGciOi...</code>	Lightly — public-scoped browser token

How to enter each (do this for all three):

1. **Key** = the variable name (e.g. `NEXT_PUBLIC_API_BASE_URL`).
2. **Value** = the value from the table.
3. **Environments** = tick **Production**, **Preview**, and **Development**.
4. Click **Add**.

⚠ Use your **real Render URL** from Part 2 Step 16 for `NEXT_PUBLIC_API_BASE_URL`. If you haven't deployed the backend yet, go back and do that first. If you want a backend-free demo, set `NEXT_PUBLIC_DATA_SOURCE=mock` instead and the site works standalone.

🖼 Screenshot placeholder: Vercel environment-variables form with the three `NEXT_PUBLIC` variables.

Step 7 — Deploy

1. Click **Deploy**. Vercel installs, builds (`next build`), and publishes.
2. When done you'll see 🎉 **Congratulations** and a live URL like `https://project-zenith.vercel.app`.
3. Click it — the globe should load.

🖼 Screenshot placeholder: Vercel deployment success screen with the live URL.

Part 5 — Connect Frontend to Backend

If you already set `NEXT_PUBLIC_API_BASE_URL` to the real Render URL in Part 4, you're connected — skip to verification. If you deployed the frontend **before** the backend (or used a placeholder), do this:

1. **Copy the Render backend URL** (Part 2, Step 16) — e.g. `https://zenith-backend.onrender.com`.
2. In **Vercel** → your project → **Settings** → **Environment Variables**.
3. Edit `NEXT_PUBLIC_API_BASE_URL` and set it to that Render URL (no trailing slash). Save.

4. **Redeploy** so the new value takes effect: **Deployments** tab → latest deployment → ... → **Redeploy**.

Why a redeploy is required: `NEXT_PUBLIC_*` variables are **baked into the JavaScript bundle at build time**, not read live. Changing the value in settings does nothing until Vercel rebuilds the site. This is normal Next.js behavior.

Part 6 — Post-Deployment Testing

Open your live Vercel URL and test each feature. (On Render Free, click around once first to wake the backend, then retest.)

#	Feature	How to test	Expected result
1	Homepage	Open the Vercel URL	3D globe + starfield + dashboard load; no blank screen
2	Search	Type a city (e.g. "Tokyo") in the search box	Suggestions appear; selecting one moves the globe & loads a report
3	Observation Score	Select any location	A 0–100 score with a Poor/Fair/Good/Excellent label and contributing factors
4	Timeline	Click <code>Now</code> → <code>+1h</code> → <code>Tonight</code> → <code>Next Week</code>	Report visibly changes (clouds, moon, events) per step
5	Visible Tonight / Planet Details	Open the Visible Tonight panel; click an object	Planets + ISS listed; clicking shows altitude/velocity/visibility window
6	Astronomical Events	View the Upcoming Events card	Sunset, twilight, moonrise/set, moon phase, next meteor shower listed with local times
7	ISS	Look for the ISS marker on the globe	Marker shows; with a valid <code>N2YO_API_KEY</code> a pass event also appears
8	Explain Tonight's Sky	Click the "Explain Tonight's Sky" button	A typewriter narration appears (Gemini if keyed, fallback otherwise) — never empty
9	API Health	Open <code>https://YOUR_BACKEND/api/health</code>	<code>{"status":"OK", ...}</code>

✅ If all pass, you are deployed and demo-ready.

Part 7 — Common Deployment Errors

Symptom	Likely cause	Fix
Render log: tsx: not found / sh: tsx: command not found	Build skipped devDependencies (where tsx lives)	Set Render Build Command to npm install --include=dev (Part 2 Step 8), then Manual Deploy
Render: Cannot find module './app' or wrong paths	Root Directory not set to backend	Render → Settings → set Root Directory = backend , redeploy
Render service builds but shows Unhealthy	Wrong/empty Health Check Path	Set it to /api/health (Part 2 Step 13)
Frontend loads but all data fails / spinners forever	NEXT_PUBLIC_API_BASE_URL wrong, missing, or has trailing slash; or you didn't redeploy after setting it	Set it to the exact Render URL (no trailing /), then Redeploy the Vercel project (Part 5)
Browser console: CORS error	Backend not reachable (it actually allows all origins via app.use(cors())), or the URL/protocol is wrong (http vs https)	Verify /api/health opens directly in the browser over https ; fix NEXT_PUBLIC_API_BASE_URL
Build failure on Vercel	next build errored	Reproduce locally with cd frontend && npm run build ; fix the reported file, push, it redeloys
Cesium assets 404 / blank globe	Cesium static files not copied	They're copied by the postinstall script during npm install . Ensure Vercel Install Command is npm install (not --omit), Root Directory = frontend ; redeploy
PORT / "port already in use" / app not binding	Hardcoded port	Don't set PORT on Render — the app reads process.env.PORT automatically. Remove any manual PORT var
API requests time out on first hit	Render Free service was asleep (cold start)	Wait ~30–60 s and retry; or hit /api/health to warm it; or upgrade to a paid always-on instance
AI narration is generic/templated	No/invalid GEMINI_API_KEY or Gemini quota exhausted (HTTP 429)	Add a valid GEMINI_API_KEY on Render. This is non-fatal — fallback narration is expected behavior
No ISS "next pass" event	Missing/invalid N2YO_API_KEY	Add a valid N2YO_API_KEY on Render. Non-fatal — live position still shows
Render service keeps sleeping during	Free plan idles after ~15 min	Ping /api/health every few minutes before/while demoing, or upgrade to

Symptom	Likely cause	Fix
demo		Starter
Vercel shows old content after an env change	<code>NEXT_PUBLIC_*</code> is build-time; cached build	Change the value, then Redeploy (Part 5). A plain refresh won't help
Missing env at runtime (feature silently off)	Forgot a variable or didn't tick the right Environment in Vercel	Re-check Part 2 Step 15 (Render) and Part 4 Step 6 (Vercel); ensure Production is ticked; redeploy

Part 8 — Production / Submission Checklist

Final pass before you submit to the hackathon.

- ☐ **GitHub repo** is pushed, up to date, and **public** (or judges are added as collaborators if private).
- ☐ **README.md** present at root, accurate, with the real GitHub URL filled in (replace the `<username>/<repository>` placeholder).
- ☐ **LICENSE** file present (MIT).
- ☐ **Blueprint / presentation PDF** included (this repo: [docs/blueprint/Juno_ProjectZenith.pdf](#)).
- ☐ **Backend URL** is live and `/api/health` returns OK — record it:
`https://_____.onrender.com`
- ☐ **Frontend (Demo) URL** is live — record it: `https://_____.vercel.app`
- ☐ **Frontend `NEXT_PUBLIC_API_BASE_URL`** points at the live backend, and the site was **redeployed** after setting it.
- ☐ **Environment variables** set on both platforms: Render → `GEMINI_API_KEY`, `N2YO_API_KEY` (optional but recommended). Vercel → `NEXT_PUBLIC_API_BASE_URL`, `NEXT_PUBLIC_DATA_SOURCE=live`, `NEXT_PUBLIC_CESIUM_ION_TOKEN` (optional).
- ☐ **No secrets committed** — `backend/.env` and `frontend/.env.local` are NOT in the repo.
- ☐ **Screenshots** added to `assets/screenshots/` and referenced in the README.
- ☐ **All 9 features in Part 6** tested on the **live** URL (not just locally).
- ☐ Backend **warmed up** right before judging (Free plan cold-start mitigation).

 When every box is ticked, your live demo URL is ready to hand to the judges.

Appendix — Accounts & API Keys You May Need

Service	Needed for	Cost	Where to get it
GitHub	Hosting the code; logging into Render & Vercel	Free	https://github.com
Render	Backend hosting	Free tier available	https://render.com
Vercel	Frontend hosting	Free (Hobby)	https://vercel.com

Service	Needed for	Cost	Where to get it
Google Gemini	Live AI Sky Narration (<code>GEMINI_API_KEY</code>)	Free tier	https://aistudio.google.com/apikey
N2YO	Real ISS pass windows (<code>N2YO_API_KEY</code>)	Free	https://www.n2yo.com/api/ (register → Profile → API)
Cesium ion	Hi-res globe imagery (<code>NEXT_PUBLIC_CESIUM_ION_TOKEN</code>)	Free tier	https://ion.cesium.com/tokens

Keyless data sources (no account needed — already wired and working): NASA JPL Horizons, Open-Meteo (weather + geocoding), Open Notify (ISS position), CelesTrak (ISS TLE), BigDataCloud (reverse geocode). This means the app produces **real data even with zero API keys** — the keyed services above only unlock the AI narration, ISS pass windows, and premium imagery.